

```

4  {
5      int fd1, fd2;
6      char c;
7
8      fd1 = Open("foobar.txt", O_RDONLY, 0);
9      fd2 = Open("foobar.txt", O_RDONLY, 0);
10     Read(fd2, &c, 1);
11     Dup2(fd2, fd1);
12     Read(fd1, &c, 1);
13     printf("c = %c\n", c);
14     exit(0);
15 }

```

10.10 标准 I/O

C 语言定义了一组高级输入输出函数，称为标准 I/O 库，为程序员提供了 Unix I/O 的较高级别的替代。这个库(libc)提供了打开和关闭文件的函数(fopen 和 fclose)、读和写字节的函数(fread 和 fwrite)、读和写字符串的函数(fgets 和 fputs)，以及复杂的格式化的 I/O 函数(scanf 和 printf)。

标准 I/O 库将一个打开的文件模型化为一个流。对于程序员而言，一个流就是一个指向 FILE 类型的结构的指针。每个 ANSI C 程序开始时都有三个打开的流 stdin、stdout 和 stderr，分别对应于标准输入、标准输出和标准错误：

```

#include <stdio.h>
extern FILE *stdin; /* Standard input (descriptor 0) */
extern FILE *stdout; /* Standard output (descriptor 1) */
extern FILE *stderr; /* Standard error (descriptor 2) */

```

类型为 FILE 的流是对文件描述符和流缓冲区的抽象。流缓冲区的目的和 RIO 读缓冲区的一样：就是使开销较高的 Linux I/O 系统调用的数量尽可能得小。例如，假设我们有一个程序，它反复调用标准 I/O 的 getc 函数，每次调用返回文件的下一个字符。当第一次调用 getc 时，库通过调用一次 read 函数来填充流缓冲区，然后将缓冲区中的第一个字节返回给应用程序。只要缓冲区中还有未读的字节，接下来对 getc 的调用就能直接从流缓冲区得到服务。

10.11 综合：我该使用哪些 I/O 函数？

图 10-16 总结了我们在这一章里讨论过的各种 I/O 包。

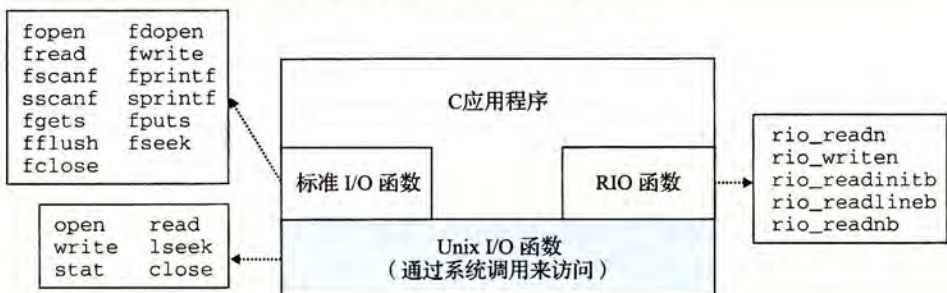


图 10-16 Unix I/O、标准 I/O 和 RIO 之间的关系

Unix I/O 模型是在操作系统内核中实现的。应用程序可以通过诸如 `open`、`close`、`lseek`、`read`、`write` 和 `stat` 这样的函数来访问 Unix I/O。较高级别的 RIO 和标准 I/O 函数都是基于(使用)Unix I/O 函数来实现的。RIO 函数是专为本书开发的 `read` 和 `write` 的健壮的包装函数。它们自动处理不足值, 并且为读文本行提供一种高效的带缓冲的方法。标准 I/O 函数提供了 Unix I/O 函数的一个更加完整的带缓冲的替代品, 包括格式化的 I/O 例程, 如 `printf` 和 `scanf`。

那么, 在你的程序中该使用这些函数中的哪一个呢? 下面是一些基本的指导原则:

- G1: 只要有可能就使用标准 I/O。对磁盘和终端设备 I/O 来说, 标准 I/O 函数是首选方法。大多数 C 程序员在其整个职业生涯中只使用标准 I/O, 从不受较低级的 Unix I/O 函数的困扰(可能 `stat` 除外, 因为在标准 I/O 库中没有与它对应的函数)。只要可能, 我们建议你也这样做。
- G2: 不要使用 `scanf` 或 `rio_readlineb` 来读二进制文件。像 `scanf` 或 `rio_readlineb` 这样的函数是专门设计来读取文本文件的。学生通常会犯的一个错误就是用这些函数来读取二进制文件, 这就使得他们的程序出现了诡异莫测的失败。比如, 二进制文件可能散布着很多 `0xa` 字节, 而这些字节又与终止文本行无关。
- G3: 对网络套接字的 I/O 使用 RIO 函数。不幸的是, 当我们试着将标准 I/O 用于网络的输入输出时, 出现了一些令人讨厌的问题。如同我们将在 11.4 节所见, Linux 对网络的抽象是一种称为套接字的文件类型。就像所有的 Linux 文件一样, 套接字由文件描述符来引用, 在这种情况下称为套接字描述符。应用程序进程通过读写套接字描述符来与运行在其他计算机的进程实现通信。

标准 I/O 流, 从某种意义上而言是全双工的, 因为程序能够在同一个流上执行输入和输出。然而, 对流的限制和对套接字的限制, 有时候会互相冲突, 而又极少有文档描述这些现象:

- 限制一: 跟在输出函数之后的输入函数。如果中间没有插入对 `fflush`、`fseek`、`fsetpos` 或者 `rewind` 的调用, 一个输入函数不能跟随在一个输出函数之后。`fflush` 函数清空与流相关的缓冲区。后三个函数使用 Unix I/O `lseek` 函数来重置当前的文件位置。
- 限制二: 跟在输入函数之后的输出函数。如果中间没有插入对 `fseek`、`fsetpos` 或者 `rewind` 的调用, 一个输出函数不能跟随在一个输入函数之后, 除非该输入函数遇到了一个文件结束。

这些限制给网络应用带来了一个问题, 因为对套接字使用 `lseek` 函数是非法的。对流 I/O 的第一个限制能够通过采用在每个输入操作前刷新缓冲区这样的规则来满足。然而, 要满足第二个限制的唯一办法是, 对同一个打开的套接字描述符打开两个流, 一个用来读, 一个用来写:

```
FILE *fpin, *fpout;
```

```
fpin = fdopen(sockfd, "r");
fpout = fdopen(sockfd, "w");
```

但是这种方法也有问题, 因为它要求应用程序在两个流上都要调用 `fclose`, 这样才能释放与每个流相关联的内存资源, 避免内存泄漏:

```
fclose(fpin);
fclose(fpout);
```


这些操作中的每一个都试图关闭同一个底层的套接字描述符，所以第二个 `close` 操作就会失败。对顺序的程序来说，这并不是问题，但是在一个线程化的程序中关闭一个已经关闭了的描述符是会导致灾难的(见 12.7.4 节)。

因此，我们建议你在网络套接字上不要使用标准 I/O 函数来进行输入和输出，而要使用健壮的 RIO 函数。如果你需要格式化的输出，使用 `sprintf` 函数在内存中格式化一个字符串，然后用 `rio_writen` 把它发送到套接口。如果你需要格式化输入，使用 `rio_readlineb` 来读一个完整的文本行，然后用 `sscanf` 从文本行提取不同的字段。

10.12 小结

Linux 提供了少量的基于 Unix I/O 模型的系统级函数，它们允许应用程序打开、关闭、读和写文件，提取文件的元数据，以及执行 I/O 重定向。Linux 的读和写操作会出现不足值，应用程序必须能正确地预计和处理这种情况。应用程序不应直接调用 Unix I/O 函数，而应该使用 RIO 包，RIO 包通过反复执行读写操作，直到传送完所有的请求数据，自动处理不足值。

Linux 内核使用三个相关的数据结构来表示打开的文件。描述符表中的表项指向打开文件表中的表项，而打开文件表中的表项又指向 v-node 表中的表项。每个进程都有它自己单独的描述符表，而所有的进程共享同一个打开文件表和 v-node 表。理解这些结构的一般组成就能使我们清楚地理解文件共享和 I/O 重定向。

标准 I/O 库是基于 Unix I/O 实现的，并提供了一组强大的高级 I/O 例程。对于大多数应用程序而言，标准 I/O 更简单，是优于 Unix I/O 的选择。然而，因为对标准 I/O 和网络文件的一些相互不兼容的限制，Unix I/O 比之标准 I/O 更该适用于网络应用程序。

参考文献说明

Kerrisk 撰写了关于 Unix I/O 和 Linux 文件系统的综述 [62]。Stevens 编写了 Unix I/O 的标准参考文献 [111]。Kernighan 和 Ritchie 对于标准 I/O 函数给出了清晰而完整的讨论 [61]。

家庭作业

* 10.6 下面程序的输出是什么？

```
1  #include "csapp.h"
2
3  int main()
4  {
5      int fd1, fd2;
6
7      fd1 = Open("foo.txt", O_RDONLY, 0);
8      fd2 = Open("bar.txt", O_RDONLY, 0);
9      Close(fd2);
10     fd2 = Open("baz.txt", O_RDONLY, 0);
11     printf("fd2 = %d\n", fd2);
12     exit(0);
13 }
```

* 10.7 修改图 10-5 中所示的 `cpfile` 程序，使得它用 RIO 函数从标准输入复制到标准输出，一次 `MAXBUF` 个字节。

** 10.8 编写图 10-10 中的 `statcheck` 程序的一个版本，叫做 `fstatcheck`，它从命令行上取得一个描述符数字而不是文件名。

** 10.9 考虑下面对作业题 10.8 中的 `fstatcheck` 程序的调用：

```
linux> fstatcheck 3 < foo.txt
```

你可能会预想这个对 `fstatcheck` 的调用将提取和显示文件 `foo.txt` 的元数据。然而，当我们在